TP 4 : un pipeline temps réel avec Kafka

Travail en binôme. Plateformes : le **Kafka de l'enseignant**, votre **projet Google Cloud** (Cloud Shell, Cloud Storage, cluster Spark, BigQuery).
Prérequis : cours de la séance 4 ; un projet GCP **avec facturation** (crédits Éducation ou essai)

> Contexte : ShopFlow publie désormais ses paiements et sa navigation en temps réel dans Kafka. L'équipe fraude veut voir **en moins de 3 minutes** les pics de refus par pays et les adresses IP qui essaient beaucoup de cartes différentes (test de cartes volées). Le pipeline complet vous est fourni : consommateur Python vers le lac (bronze), agrégats Spark Structured Streaming vers BigQuery, tableau de bord.

Votre travail consiste à **observer** ce qui se passe et à **expliquer** vos observations en répondant aux questions.

TP 4 : un pipeline temps réel avec Kafka (suite)

Le pipeline du TP 4 et ses scripts

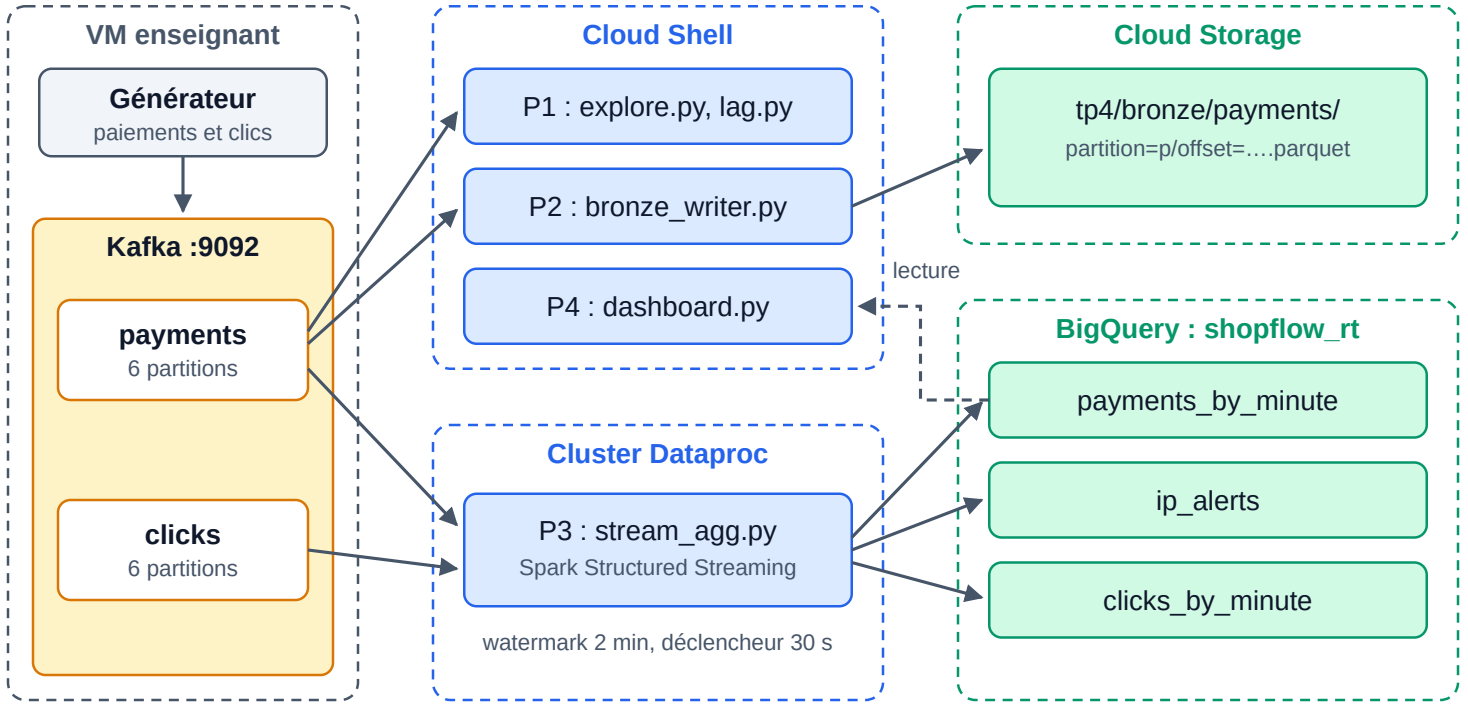


Schéma : chaque script du TP (P1 à P4) lit les topics de la VM enseignant et écrit soit le bronze sur Cloud Storage, soit les trois tables de `shopflow_rt` *lues par le tableau de bord.*

Ce que l'enseignant vous donne

Élément	Valeur
Adresse Kafka (`KAFKA_BOOTSTRAP`)	`IP_KAFKA:9092`
Identifiant et mot de passe	.....
Topics	`payments` et `clicks` , 6 partitions chacun
Votre identifiant de binôme (`BINOME`)	`à vous de le définir` ...

Le flux génère les données ShopFlow. Il contient **volontairement** environ 1 % de doublons et 2 % de messages en retard.

Format d'un message de `payments` (clé : `card_id` , ou `c<customer_id>` pour un paiement sans carte) :

Ce que l'enseignant vous donne (suite)


```
{"payment_id": 1834512, "order_id": 1801233, "customer_id": 41217, "card_id": 412171,
 "amount": 89.9, "currency": "EUR", "payment_method": "card", "card_country": "FR",
 "ip": "82.64.12.7", "status": "authorized", "shipping_country": "FR", "device": "mobile",
 "event_ts": "2026-11-23T09:14:03.512000+00:00", "orig_ts": "2025-11-04T18:27:11+00:00"}
```


`event_ts` heure du génération du message : c'est elle qu'on utilise pour les fenêtres de ce TP.

Format d'un message de `clicks` (clé : `session_id`) : `event_id`, `session_id`, `customer_id`, `event_type`, `page`, `product_id`, `device`, `ip`, `event_ts`, `orig_ts`.

Partie 0 : environnement

Ouvrez **Cloud Shell** (bouton `>_` de la console Google Cloud). Tout le code Python de ce TP y tourne.

Étape 1. Environnement Python (une seule fois) :

```
python3 -m venv ~/tp4 && source ~/tp4/bin/activate
pip install confluent-kafka pandas pyarrow gcsfs streamlit google-cloud-bigquery db-dtypes
cd ~/tp4
```

Étape 1 bis. Autoriser vos scripts Python à utiliser votre compte Google. Les commandes `gcloud` de Cloud Shell sont déjà connectées, mais pas les bibliothèques Python (Cloud Storage, BigQuery) :

```
gcloud auth application-default login
```

Partie 0 : environnement (suite)

Étape 2. Vos variables. Dans le bloc ci-dessous, remplacez d'abord `IP_KAFKA`, `MOT_DE_PASSE` et `id_binome` par les valeurs du tableau, **puis** collez-le. Il crée le fichier `env.sh`.

```
cat > ~/tp4/env.sh <<'EOF'
export KAFKA_BOOTSTRAP=IP_KAFKA:9092
export KAFKA_USER=etudiant
export KAFKA_PASSWORD='MOT_DE_PASSE'
export BINOME=id_binome
export PROJECT=$(gcloud config get-value project 2>/dev/null)
export BUCKET=gs://$PROJECT-shopflow
EOF
source ~/tp4/env.sh
echo "Kafka=$KAFKA_BOOTSTRAP  binôme=$BINOME  projet=$PROJECT  bucket=$BUCKET"
```

Le bucket a normalement été créé au TP 3. Cette commande le crée seulement s'il manque :

Partie 0 : environnement (suite)

```
gcloud storage buckets describe $BUCKET >/dev/null 2>&1 || gcloud storage buckets create $BUCKET --location=europe-west1
```

> **Dans chaque nouvel onglet Cloud Shell** (bouton `+`), commencez toujours par :
>
> ```
> cd ~/tp4 && source bin/activate && source env.sh
> ```

Étape 3. La configuration Kafka commune à tous les scripts :

Partie 0 : environnement (suite)

```
cat > ~/tp4/kafka_conf.py <<'EOF'
import os

def conf(**extra) -> dict:
    """Configuration commune des clients Kafka du TP."""
    c = {
        "bootstrap.servers": os.environ["KAFKA_BOOTSTRAP"],
        "security.protocol": "SASL_PLAINTEXT",
        "sasl.mechanism": "PLAIN",
        "sasl.username": os.environ["KAFKA_USER"],
        "sasl.password": os.environ["KAFKA_PASSWORD"],
    }
    c.update(extra)
    return c
EOF
```

Étape 4. Test de connexion. Le port doit être ouvert, puis la liste des topics doit contenir `clicks` et `payments` :

Partie 0 : environnement (suite)

```
timeout 5 bash -c "</dev/tcp/${KAFKA_BOOTSTRAP%:*}/${KAFKA_BOOTSTRAP#*:}" && echo "port OUVERT" || echo "port FERMÉ : vérifiez KAFKA_BOOTSTRAP"
python -c "from confluent_kafka.admin import AdminClient; from kafka_conf import conf; print(sorted(AdminClient(conf()).list_topics(timeout=10).topics))"
```

Partie 1 : consommateurs, partitions, groupes, lag

1.A Lire le flux

Créez le script `explore.py`. Il affiche chaque paiement reçu : partition, offset, clé, heure, montant et statut. Un second argument, facultatif, ajoute une pause (en secondes) après chaque message, pour simuler un consommateur lent.

1.A Lire le flux (suite)

```
cat > ~/tp4/explore.py <<'EOF'
import json
import sys
import time
from confluent_kafka import Consumer
from kafka_conf import conf

group = sys.argv[1]
pause = float(sys.argv[2]) if len(sys.argv) > 2 else 0.0

consumer = Consumer(conf(**{
    "group.id": group,
    "auto.offset.reset": "latest",     # sans offset commité : on part de la fin
    "enable.auto.commit": True,
}))

def on_assign(c, partitions):
    print("attribuées :", sorted(p.partition for p in partitions), flush=True)

def on_revoke(c, partitions):
    print("retirées   :", sorted(p.partition for p in partitions), flush=True)
```

1.A Lire le flux (suite)

```
consumer.subscribe(["payments"], on_assign=on_assign, on_revoke=on_revoke)
try:
    while True:
        msg = consumer.poll(1.0)
        if msg is None:
            continue
        if msg.error():
            print("erreur :", msg.error(), flush=True)
            continue
        v = json.loads(msg.value())
        print(f"p={msg.partition()} off={msg.offset()} key={msg.key().decode()} "
              f"ts={v['event_ts']} {v['amount']:>8} {v['status']}", flush=True)
        if pause:
            time.sleep(pause)
except KeyboardInterrupt:
    pass
finally:
    consumer.close()
EOF
```

1.A Lire le flux (suite)

Lancez-le pendant **deux minutes**, en gardant une copie de l'affichage dans `explore.log`, puis arrêtez-le avec `Ctrl+C` :

```
python explore.py $BINOME-explore | tee explore.log
```

Cherchez les cartes vues plusieurs fois, puis affichez tous les messages de la première :

```
grep -o "key=[^ ]*" explore.log | sort | uniq -c | sort -rn | head -5
CLE=$(grep -o "key=[^ ]*" explore.log | sort | uniq -c | sort -rn | head -1 | awk '{print $2}')
grep "$CLE " explore.log
```

❓ Q1. Les messages de cette carte (`card_id`) sont-ils toujours dans la même partition ? Pourquoi est-ce important pour calculer « le nombre de paiements de cette carte dans la dernière heure » ?

1.B Groupes et rééquilibrage

Vous allez faire varier le nombre de membres du groupe `$BINOME-explore` et regarder les lignes `attribuées` / `retirées`.

1. **1 membre.** Dans l'onglet 1 : `python explore.py $BINOME-explore`. Notez les partitions attribuées.
2. **2 membres.** Ouvrez un onglet 2 (`+`, puis la commande d'entrée de la partie 0) et lancez la même commande. Regardez ce qui s'affiche **dans les deux onglets**.
3. **3 membres.** Même chose dans un onglet 3.
4. **7 membres.** Dans un onglet 4, lancez 4 membres de plus en arrière-plan, puis affichez leurs attributions :

```
cd ~/tp4 && source bin/activate && source env.sh
for i in 4 5 6 7; do python explore.py $BINOME-explore > membre$i.log 2>&1 & done
sleep 20; grep -H "attribuées\|retirées" membre*.log | tail -8
```

1.B Groupes et rééquilibrage (suite)

Regardez aussi les onglets 1 à 3.

5. **Un membre part.** Arrêtez l'onglet 3 avec `Ctrl+C`, attendez 10 secondes, puis relancez `grep -H "attribuées\|retirées" membre*.log | tail -8` dans l'onglet 4.
6. **Un autre groupe.** Dans l'onglet 3, lancez `python explore.py $BINOME-autre` et comparez ses messages à ceux de l'onglet 1.
7. **Nettoyage.** Arrêtez tout : `Ctrl+C` dans les onglets 1, 2 et 3, puis dans l'onglet 4 : `pkill -f "explore.py $BINOME-explore"`.

❓ Q2. Combien de partitions reçoit chaque consommateur avec 1, 2, 3 puis 7 membres ? Que fait le 7e ? Que se passe-t-il pendant un rééquilibrage ?

❓ Q3. Le consommateur du groupe `$BINOME-autre` voit-il les mêmes messages que ceux du premier groupe ? Pourquoi ?

1.C Mesurer le lag

Créez `lag.py`. Pour un groupe donné, il affiche par partition de `payments` l'offset commité du groupe, l'offset haut (dernier message écrit + 1) et leur différence, le **lag**.

1.C Mesurer le lag (suite)

```
cat > ~/tp4/lag.py <<'EOF'
import sys
from confluent_kafka import Consumer, TopicPartition
from kafka_conf import conf

group = sys.argv[1]
c = Consumer(conf(**{"group.id": group, "enable.auto.commit": False}))
tps = [TopicPartition("payments", p) for p in range(6)]
committed = c.committed(tps, timeout=10)          # offsets commités du groupe
total = 0
print(f"groupe {group}")
print(f"{'part':>4} {'commité':>10} {'haut':>10} {'lag':>8}")
for tp in committed:
    low, high = c.get_watermark_offsets(TopicPartition(tp.topic, tp.partition), timeout=10)
    # un offset négatif veut dire « jamais commité » : tout ce qui reste dans la partition est en retard
    lag = high - tp.offset if tp.offset >= 0 else high - low
    total += lag
    print(f"{tp.partition:>4} {tp.offset:>10} {high:>10} {lag:>8}")
print("lag total :", total)
c.close()
EOF
```

1.C Mesurer le lag (suite)

Une partition de payments vue par lag.py



c.committed(tps) → offset commité = prochain message à lire (dernier lu + 1)

c.get_watermark_offsets(tp) → (bas, haut) : premier offset conservé, prochain à écrire

Ici « watermark » désigne les bornes de la partition, sans rapport avec le watermark de Spark.
Offset commité négatif : le groupe n'a jamais commité sur cette partition.

Schéma : `lag.py` compare l'offset commité du groupe à l'offset haut de la partition ; la différence est le nombre de messages pas encore lus.

1.C Mesurer le lag (suite)

Lancez un consommateur **lent** (une pause de 0,5 s après chaque message) dans l'onglet 1 :

```
python explore.py $BINOME-lent 0.5
```

et suivez son lag pendant 3 à 4 minutes dans l'onglet 2 :

```
watch -n 5 python lag.py $BINOME-lent
```

Pour comparer, ajoutez un 2e consommateur lent du même groupe dans l'onglet 3 (`python explore.py $BINOME-lent 0.5`) et regardez à nouveau le lag pendant 2 minutes.

❓ Q4. Comment évolue le lag avec un puis deux consommateurs lents ? Que faudrait-il faire en production si le lag d'un groupe croît en permanence ? Quelle est la limite de cette solution avec 6 partitions ?

1.C Mesurer le lag (suite)

Arrêtez tous les consommateurs de la partie 1 (`Ctrl+C` partout) avant de continuer.

Partie 2 : écrire le bronze sans perte ni doublon

Le script `bronze_writer.py` écrit `payments` dans `$BUCKET/tp4/bronze/payments/` en Parquet, par lots, et **ne perd rien et ne duplique rien même s'il plante**. Voici comment il s'y prend :

1. `enable.auto.commit` est à `False` : c'est le script qui commite, au bon moment.
2. Il accumule les messages **par partition** jusqu'à 2 000 messages ou 30 secondes.
3. Il écrit **un fichier par partition et par lot**, nommé par la partition et le **premier offset** du lot : `partition=3/offset=000000012345.parquet`. Si le lot est rejoué après un plantage, il réécrit **le même fichier** : la sortie est idempotente.
4. Il commite les offsets **après** l'écriture, avec pour chaque partition l'offset du dernier message **+ 1** (le prochain à lire).

Partie 2 : écrire le bronze sans perte ni doublon (suite)

Un lot de la partition 3 : écrire, puis commiter

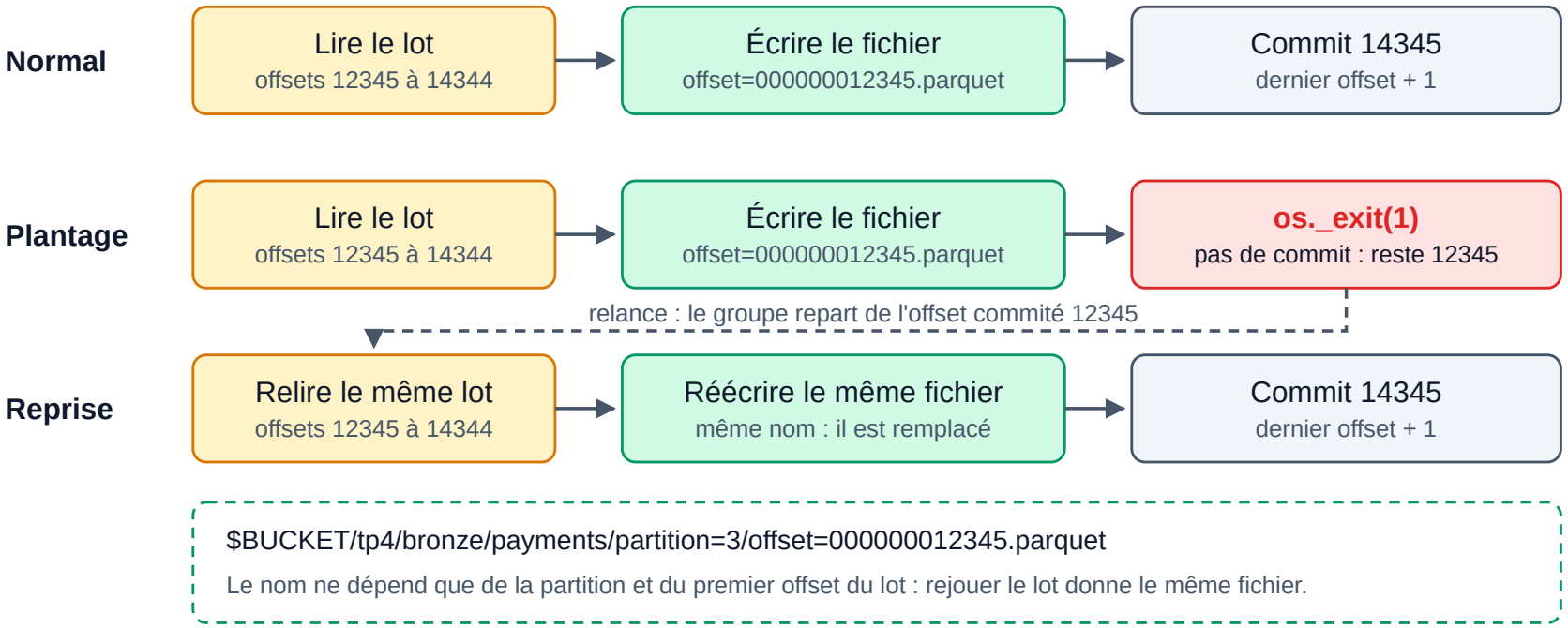


Schéma : un plantage entre l'écriture et le commit fait relire le même lot à partir de l'offset commité, et le fichier réécrit porte le même nom que le premier.

Partie 2 : écrire le bronze sans perte ni doublon (suite)

```
cat > ~/tp4/bronze_writer.py <<'EOF'
import json
import os
import time
from collections import defaultdict

import gcsfs
import pandas as pd
from confluent_kafka import Consumer, TopicPartition
from kafka_conf import conf

fs = gcsfs.GCSFileSystem()           # accès à Cloud Storage avec les identifiants de Cloud Shell
BUCKET = os.environ["BUCKET"]
OUT = f"{BUCKET}/tp4/bronze/payments"
MAX_MSGS, MAX_SECONDS = 2000, 30
CRASH = os.environ.get("CRASH_AFTER_WRITE") == "1"   # pour tester la reprise

consumer = Consumer(conf(**{
    "group.id": f"{os.environ['BINOME']}-bronze",
    "auto.offset.reset": "earliest",
    "enable.auto.commit": False,
}))
```

Partie 2 : écrire le bronze sans perte ni doublon (suite)

```
consumer.subscribe(["payments"])

buffers = defaultdict(list)          # partition -> liste de (offset, message décodé)
last_flush = time.time()


def flush():
    """Écrit un fichier Parquet par partition, PUIS commite les offsets."""
    to_commit = []
    for p, msgs in buffers.items():
        if not msgs:
            continue
        first, last = msgs[0][0], msgs[-1][0]
        df = pd.DataFrame([v for _, v in msgs])
        df["kafka_partition"] = p
        df["kafka_offset"] = [o for o, _ in msgs]
        path = f"{OUT}/partition={p}/offset={first:012d}.parquet"
        with fs.open(path, "wb") as f:               # écriture vers gs:// via gcsfs
            df.to_parquet(f, index=False)
        print(f"écrit {path} ({len(df)} lignes, offsets {first} à {last})", flush=True)
        if CRASH:
            print("plantage simulé entre l'écriture et le commit", flush=True)
```

Partie 2 : écrire le bronze sans perte ni doublon (suite)

```
            os._exit(1)
        to_commit.append(TopicPartition("payments", p, last + 1))
        msgs.clear()
    if to_commit:
        consumer.commit(offsets=to_commit, asynchronous=False)
        print("commité :", {tp.partition: tp.offset for tp in to_commit}, flush=True)


try:
    while True:
        for msg in consumer.consume(num_messages=500, timeout=1.0):
            if msg.error():
                print("erreur :", msg.error(), flush=True)
                continue
            buffers[msg.partition()].append((msg.offset(), json.loads(msg.value())))
        if any(len(b) >= MAX_MSGS for b in buffers.values()) or time.time() - last_flush > MAX_SECONDS:
            flush()
            last_flush = time.time()
except KeyboardInterrupt:
    pass
finally:
    consumer.close()
```

Partie 2 : écrire le bronze sans perte ni doublon (suite)

```
EOF
```

Et un script de contrôle, `check_bronze.py`, qui relit tout le bronze et compte, par partition, les offsets manquants (trous) et en double, puis les `payment_id` en double :

Partie 2 : écrire le bronze sans perte ni doublon (suite)

```
cat > ~/tp4/check_bronze.py <<'EOF'
import logging
import os
import gcsfs
import pyarrow.parquet as pq

logging.getLogger("asyncio").setLevel(logging.CRITICAL)
fs = gcsfs.GCSFileSystem()
root = os.environ["BUCKET"].removeprefix("gs://") + "/tp4/bronze/payments"
df = pq.read_table(root, filesystem=fs).to_pandas()
g = df.groupby("kafka_partition").kafka_offset.agg(["min", "max", "count", "nunique"])
g["trous"] = g["max"] - g["min"] + 1 - g["nunique"]
g["offsets_en_double"] = g["count"] - g["nunique"]
print(g)
print("lignes :", len(df), " payment_id distincts :", df.payment_id.nunique(),
      " payment_id en double :", len(df) - df.payment_id.nunique())
EOF
```

Partie 2 : écrire le bronze sans perte ni doublon (suite)

Test 1. Fonctionnement normal. Lancez le writer pendant 2 minutes, arrêtez-le (`Ctrl+C`), puis listez les fichiers et contrôlez :

```
python bronze_writer.py
gcloud storage ls -r $BUCKET/tp4/bronze/payments/ | head -20
python check_bronze.py
```

Test 2. Plantage entre l'écriture et le commit. Le premier lancement écrit un fichier puis « plante » ; le second reprend normalement. Laissez tourner le second 1 minute, arrêtez-le, puis contrôlez à nouveau :

```
CRASH_AFTER_WRITE=1 python bronze_writer.py
python bronze_writer.py
python check_bronze.py
```

Partie 2 : écrire le bronze sans perte ni doublon (suite)

Comparez la ligne `écrit …` du plantage avec les premières lignes `écrit …` de la reprise.

❓ Q5. Quelle garantie de livraison ce consommateur offre-t-il ? Que se passerait-il si on commitait **avant** d'écrire ?
❓ Q6. Après la reprise, y a-t-il des offsets en double ? Et des `payment_id` en double ? Expliquez la différence : quel mécanisme protège de quoi, et où faut-il dédupliquer les doublons restants (pensez à la séance 2) ?

Relancez `python bronze_writer.py` dans un onglet dédié et **laissez-le tourner** pendant la suite.

Partie 3 : Spark Structured Streaming vers BigQuery

3.A Préparer

Dataset BigQuery de destination :

```
bq --location=EU mk --dataset $PROJECT:shopflow_rt
bq ls                                  # shopflow_rt doit apparaître
```

Droits du cluster : les machines Dataproc agissent avec le **compte de service par défaut** du projet. Dans le projet, il n'a aucun droit sur BigQuery ni sur vos buckets :

```
SA=$(gcloud projects describe $PROJECT --format='value(projectNumber)')-compute@developer.gserviceaccount.com
for ROLE in roles/dataproc.worker roles/storage.objectAdmin roles/bigquery.dataEditor roles/bigquery.jobUser; do
  gcloud projects add-iam-policy-binding $PROJECT --member=serviceAccount:$SA --role=$ROLE --condition=None > /dev/null
done
gcloud projects get-iam-policy $PROJECT --flatten=bindings --filter="bindings.members:$SA" --format='value(bindings.role)'
```

3.A Préparer (suite)

La dernière commande doit lister les 4 rôles. Comptez 1 à 2 minutes avant que les droits s'appliquent.

Cluster Spark (même commande qu'au TP 3, avec deux ajouts : des IP publiques pour joindre le Kafka de l'enseignant, et une durée de vie maximale, car un job de streaming ne s'arrête jamais seul et le cluster n'est donc jamais inactif). La création prend 2 à 3 minutes :

```
gcloud dataproc clusters create shopflow-spark \
  --region=europe-west1 \
  --image-version=3.0-debian13 \
  --master-machine-type=n2-standard-4 --master-boot-disk-size=100GB \
  --num-workers=2 --worker-machine-type=n2-standard-4 --worker-boot-disk-size=100GB \
  --enable-component-gateway --public-ip-address \
  --delete-max-idle=30m --delete-max-age=4h
```

3.B Le job

Le job `stream_agg.py` lit les deux topics, décode le JSON, supprime les doublons de `payment_id`, et calcule trois agrégats écrits en continu dans BigQuery :

Table	Contenu	Fenêtre
`payments_by_minute`	paiements, refus et montant par pays de la carte	1 minute
`ip_alerts`	par IP : paiements, refus, nombre approximatif de cartes distinctes (≥ 3 seulement)	glissante : 10 minutes, toutes les minutes
`clicks_by_minute`	événements de navigation par type	1 minute

3.B Le job (suite)

```
cat > ~/tp4/stream_agg.py <<'EOF'
import sys
from pyspark.sql import SparkSession, functions as F, types as T

bootstrap, user, password, bucket = sys.argv[1:5]       # bucket sans le préfixe gs://
spark = (SparkSession.builder.appName("shopflow-stream")
         .config("spark.sql.session.timeZone", "UTC")
         .config("spark.sql.shuffle.partitions", "6")
         .getOrCreate())


def kafka_stream(topic):
    return (spark.readStream.format("kafka")
            .option("kafka.bootstrap.servers", bootstrap)
            .option("kafka.security.protocol", "SASL_PLAINTEXT")
            .option("kafka.sasl.mechanism", "PLAIN")
            .option("kafka.sasl.jaas.config",
                    "org.apache.kafka.common.security.plain.PlainLoginModule required "
                    f'username="{user}" password="{password}";')
            .option("subscribe", topic)
            .option("startingOffsets", "latest")
            .load())
```

3.B Le job (suite)

```
def to_bigquery(df, table, name):
    return (df.writeStream
            .queryName(name)
            .outputMode("append")
            .format("bigquery")
            .option("table", f"shopflow_rt.{table}")
            .option("temporaryGcsBucket", bucket)
            .option("checkpointLocation", f"gs://{bucket}/tp4/checkpoints/{table}")
            .trigger(processingTime="30 seconds")
            .start())


# ---------- paiements : lecture, décodage, watermark, déduplication ----------
pay_schema = T.StructType([
    T.StructField("payment_id", T.LongType()), T.StructField("order_id", T.LongType()),
    T.StructField("customer_id", T.LongType()), T.StructField("card_id", T.LongType()),
    T.StructField("amount", T.DoubleType()), T.StructField("currency", T.StringType()),
    T.StructField("payment_method", T.StringType()), T.StructField("card_country", T.StringType()),
    T.StructField("ip", T.StringType()), T.StructField("status", T.StringType()),
    T.StructField("shipping_country", T.StringType()), T.StructField("device", T.StringType()),
```

3.B Le job (suite)

```
    T.StructField("event_ts", T.StringType()),
])

payments = (kafka_stream("payments")
            .select(F.from_json(F.col("value").cast("string"), pay_schema).alias("p"))
            .select("p.*")
            .withColumn("event_ts", F.col("event_ts").cast("timestamp"))
            .withWatermark("event_ts", "2 minutes")
            .dropDuplicatesWithinWatermark(["payment_id"]))

# ---------- agrégat 1 : paiements par minute et par pays de la carte ----------
by_country = (payments
              .groupBy(F.window("event_ts", "1 minute"),
                       F.coalesce("card_country", F.lit("??")).alias("card_country"))
              .agg(F.count("*").alias("n_payments"),
                   F.sum(F.when(F.col("status") == "declined", 1).otherwise(0)).alias("n_declined"),
                   F.round(F.sum("amount"), 2).alias("amount_sum"))
              .select(F.col("window.start").alias("window_start"), F.col("window.end").alias("window_end"),
                      "card_country", "n_payments", "n_declined", "amount_sum"))
to_bigquery(by_country, "payments_by_minute", "by_country")

# ---------- agrégat 2 : IP suspectes, fenêtre glissante de 10 min toutes les minutes ----------
```

3.B Le job (suite)

```
ip_alerts = (payments
             .groupBy(F.window("event_ts", "10 minutes", "1 minute"), "ip")
             .agg(F.count("*").alias("n_tx"),
                  F.sum(F.when(F.col("status") == "declined", 1).otherwise(0)).alias("n_declined"),
                  F.approx_count_distinct("card_id").alias("n_cards"))
             .where(F.col("n_cards") >= 3)
             .select(F.col("window.start").alias("window_start"), F.col("window.end").alias("window_end"),
                     "ip", "n_tx", "n_declined", "n_cards"))
to_bigquery(ip_alerts, "ip_alerts", "ip_alerts")

# ---------- agrégat 3 : navigation, événements par minute et par type ----------
click_schema = T.StructType([
    T.StructField("event_id", T.LongType()), T.StructField("session_id", T.LongType()),
    T.StructField("customer_id", T.LongType()), T.StructField("event_type", T.StringType()),
    T.StructField("page", T.StringType()), T.StructField("product_id", T.LongType()),
    T.StructField("device", T.StringType()), T.StructField("ip", T.StringType()),
    T.StructField("event_ts", T.StringType()),
])
clicks = (kafka_stream("clicks")
          .select(F.from_json(F.col("value").cast("string"), click_schema).alias("c"))
          .select("c.*")
          .withColumn("event_ts", F.col("event_ts").cast("timestamp"))
```

3.B Le job (suite)

```
          .withWatermark("event_ts", "2 minutes"))
clicks_by_minute = (clicks
                    .groupBy(F.window("event_ts", "1 minute"), "event_type")
                    .count()
                    .select(F.col("window.start").alias("window_start"), "event_type",
                            F.col("count").alias("n_events")))
to_bigquery(clicks_by_minute, "clicks_by_minute", "clicks")

spark.streams.awaitAnyTermination()
EOF
```

Soumission (le connecteur BigQuery est préinstallé sur l'image ; le connecteur Kafka est téléchargé au démarrage) :

3.B Le job (suite)

```
gcloud dataproc jobs submit pyspark stream_agg.py \
  --cluster=shopflow-spark --region=europe-west1 \
  --properties=spark.jars.packages=org.apache.spark:spark-sql-kafka-0-10_2.13:4.1.2 \
  -- $KAFKA_BOOTSTRAP $KAFKA_USER "$KAFKA_PASSWORD" $PROJECT-shopflow
```

La commande reste attachée au job et affiche ses journaux ; `Ctrl+C` la détache **sans** arrêter le job. Pour l'arrêter plus tard : `gcloud dataproc jobs list --region=europe-west1 --state-filter=active` puis `gcloud dataproc jobs kill JOB_ID --region=europe-west1`.

Au bout de 3 à 4 minutes, vérifiez que des lignes arrivent, dans un autre onglet :

```
bq query --use_legacy_sql=false \
  'SELECT * FROM shopflow_rt.payments_by_minute ORDER BY window_start DESC LIMIT 20'
```

3.B Le job (suite)

Pour mesurer le délai, comparez l'heure actuelle à la fin de la dernière fenêtre écrite (relancez la commande 3 ou 4 fois, à une minute d'intervalle) :

```
bq query --use_legacy_sql=false \
  'SELECT CURRENT_TIMESTAMP() AS maintenant, MAX(window_end) AS derniere_fenetre,
          TIMESTAMP_DIFF(CURRENT_TIMESTAMP(), MAX(window_end), SECOND) AS retard_s
   FROM shopflow_rt.payments_by_minute'
```

3.B Le job (suite)

Ouvrez ensuite l'**interface Spark** du job (le job doit être en cours d'exécution) :

1. Console Google Cloud : *Dataproc > Clusters >* votre cluster, onglet **Interfaces web**, lien **YARN ResourceManager**.
2. Sur la ligne de l'application `shopflow-stream` (état `RUNNING`), cliquez sur le lien **ApplicationMaster** de la dernière colonne (*Tracking UI*).
3. L'interface Spark s'ouvre, avec ses onglets en haut : *Jobs, Stages, Storage, Environment, Executors, SQL / DataFrame*, et **Structured Streaming**. Ce dernier n'apparaît qu'une fois au moins une requête de streaming démarrée.
4. Dans l'onglet **Structured Streaming**, cliquez sur l'identifiant (*Run ID*) de la requête `by_country`.

Si la ligne `shopflow-stream` est `FINISHED` ou `FAILED`, le job s'est arrêté : regardez ses journaux (`gcloud dataproc jobs describe JOB_ID --region=europe-west1`) avant de continuer.

3.B Le job (suite)

❓ Q7. Un événement de 10 h 00 m 30 s arrive : à quelle heure (environ) la ligne de la fenêtre 10 h 00 - 10 h 01 apparaît-elle dans BigQuery ? Décomposez ce délai (fenêtre, watermark, déclencheur) et comparez avec le `retard_s` mesuré.

❓ Q8. L'agrégat 2 compte les cartes distinctes avec `approx_count_distinct` et pas avec `countDistinct`. Pourquoi ? (Cherchez dans le guide Structured Streaming de Spark la liste des opérations non prises en charge.)

❓ Q9. Dans l'onglet Structured Streaming, relevez le débit d'entrée (*Input Rate*), la durée des micro-lots (*Batch Duration*) et, dans le détail de la requête, le nombre de lignes ignorées par le watermark (*Aggregated Number Of Rows Dropped By Watermark*). D'où viennent ces lignes ignorées ?

3.C Fraude visible ?

```
bq query --use_legacy_sql=false \
  'SELECT window_start, ip, n_cards, n_tx, n_declined
   FROM shopflow_rt.ip_alerts
   ORDER BY window_start DESC, n_cards DESC
   LIMIT 50'
```

Prenez l'IP qui a le plus de cartes et affichez toutes ses lignes (remplacez `x.x.x.x`) :

```
bq query --use_legacy_sql=false \
  'SELECT window_start, window_end, n_cards, n_tx, n_declined
   FROM shopflow_rt.ip_alerts WHERE ip = "X.X.X.X" ORDER BY window_start'
```

Si la table est vide, attendez quelques minutes

3.C Fraude visible ? (suite)

❓ Q10. Trouvez-vous des IP avec beaucoup de cartes et beaucoup de refus ? À quel scénario de fraude du fil rouge cela correspond-il ? Une même attaque apparaît-elle dans plusieurs lignes ? Pourquoi ?

Partie 4 : tableau de bord

Le tableau de bord **Streamlit** tourne dans Cloud Shell. Il se rafraîchit toutes les 15 secondes et affiche les paiements par pays, le taux de refus des 5 dernières minutes et les 10 IP les plus suspectes.

Partie 4 : tableau de bord (suite)

```
cat > ~/tp4/dashboard.py <<'EOF'
import os

import pandas as pd
import streamlit as st
from google.cloud import bigquery

client = bigquery.Client(project=os.environ["PROJECT"])
st.set_page_config(page_title="ShopFlow temps réel", layout="wide")
st.title("ShopFlow : paiements en temps réel")


@st.fragment(run_every="15s")
def live():
    df = client.query("""
        SELECT window_start, card_country, n_payments, n_declined
        FROM shopflow_rt.payments_by_minute
        WHERE window_start > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 MINUTE)
    """).to_dataframe()
    if df.empty:
        st.info("Pas encore de données dans shopflow_rt.payments_by_minute : patientez.")
        return
```

Partie 4 : tableau de bord (suite)

```
    # indicateur : taux de refus des 5 dernières minutes de données
    last5 = df[df.window_start > df.window_start.max() - pd.Timedelta(minutes=5)]
    rate = last5.n_declined.sum() / max(last5.n_payments.sum(), 1)
    st.metric("Taux de refus, 5 dernières minutes", f"{rate:.1%}")

    st.subheader("Paiements par minute et par pays")
    st.line_chart(df.pivot_table(index="window_start", columns="card_country",
                                 values="n_payments", aggfunc="sum"))

    # tableau : les 10 IP les plus suspectes des 30 dernières minutes
    ips = client.query("""
        SELECT ip, MAX(n_cards) AS n_cards, MAX(n_declined) AS n_declined, MAX(window_end) AS vu_le
        FROM shopflow_rt.ip_alerts
        WHERE window_end > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 MINUTE)
        GROUP BY ip ORDER BY n_cards DESC LIMIT 10
    """).to_dataframe()
    st.subheader("IP suspectes")
    st.dataframe(ips)


live()
```

Partie 4 : tableau de bord (suite)

```
EOF
streamlit run dashboard.py --server.port 8080 --server.enableCORS false --server.enableXsrfProtection false
```

Ouvrez-le avec le bouton **Aperçu sur le Web > Prévisualiser sur le port 8080** de Cloud Shell.

Fin de séance : tout arrêter

```
gcloud dataproc jobs list --region=europe-west1 --state-filter=active
gcloud dataproc jobs kill JOB_ID --region=europe-west1        # JOB_ID : la première colonne de la commande précédente
gcloud dataproc clusters delete shopflow-spark --region=europe-west1
pkill -f bronze_writer.py; pkill -f streamlit
```